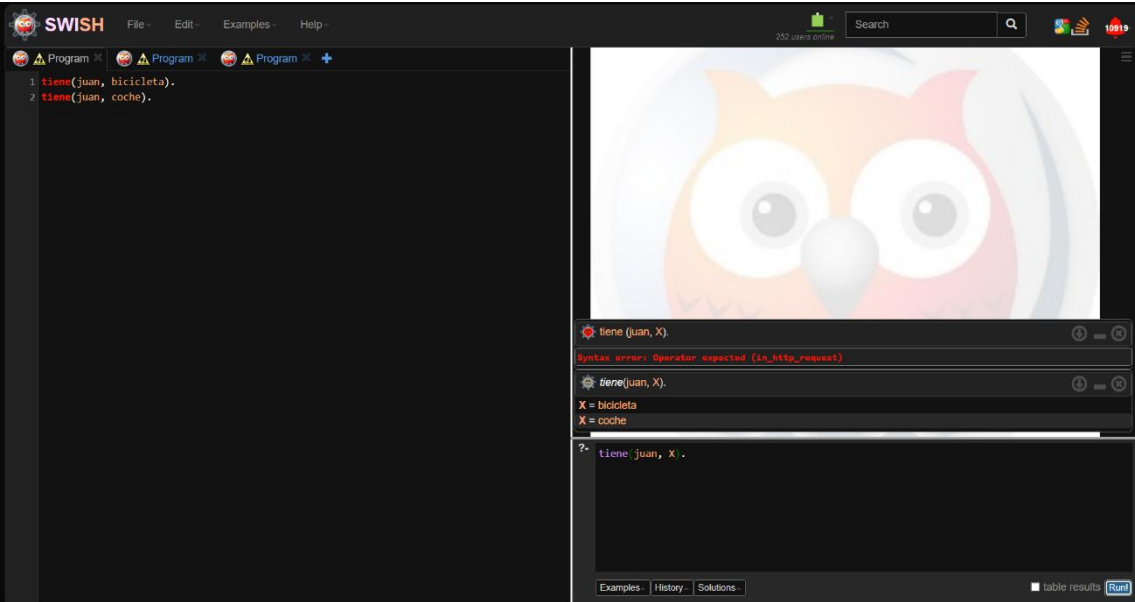


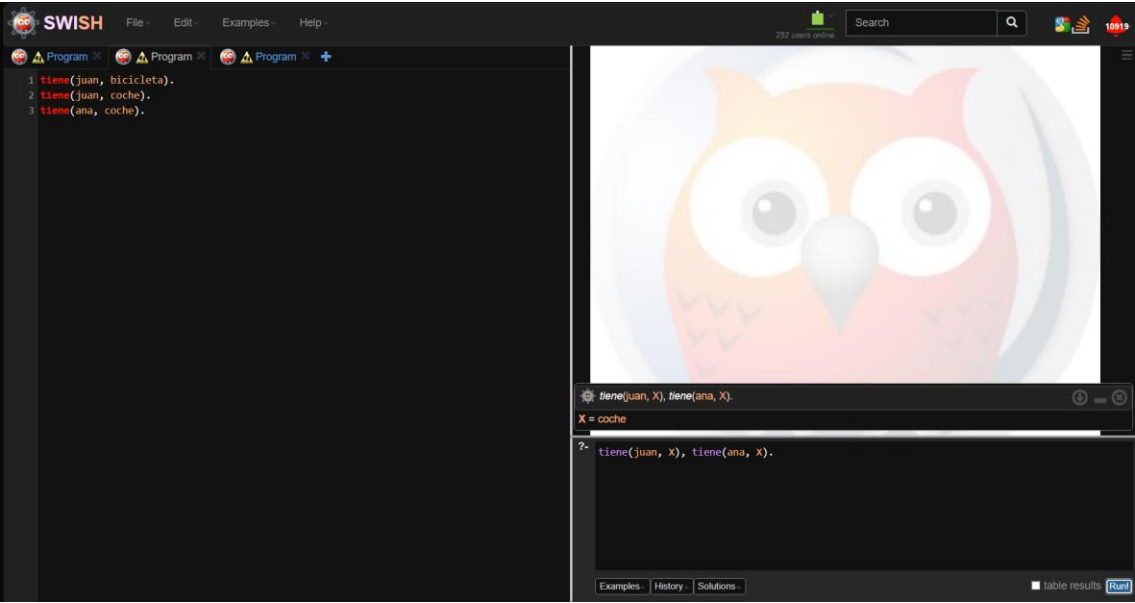
Proyecto Lenguaje de programación

08/09/2026

1

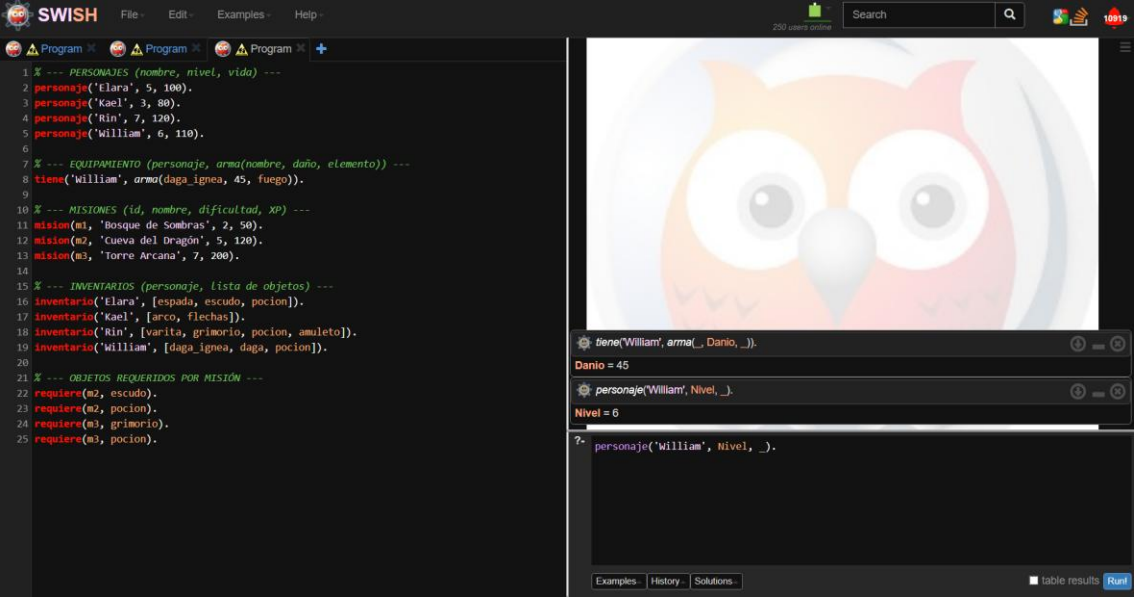


2



7

Proyecto practicas en clase

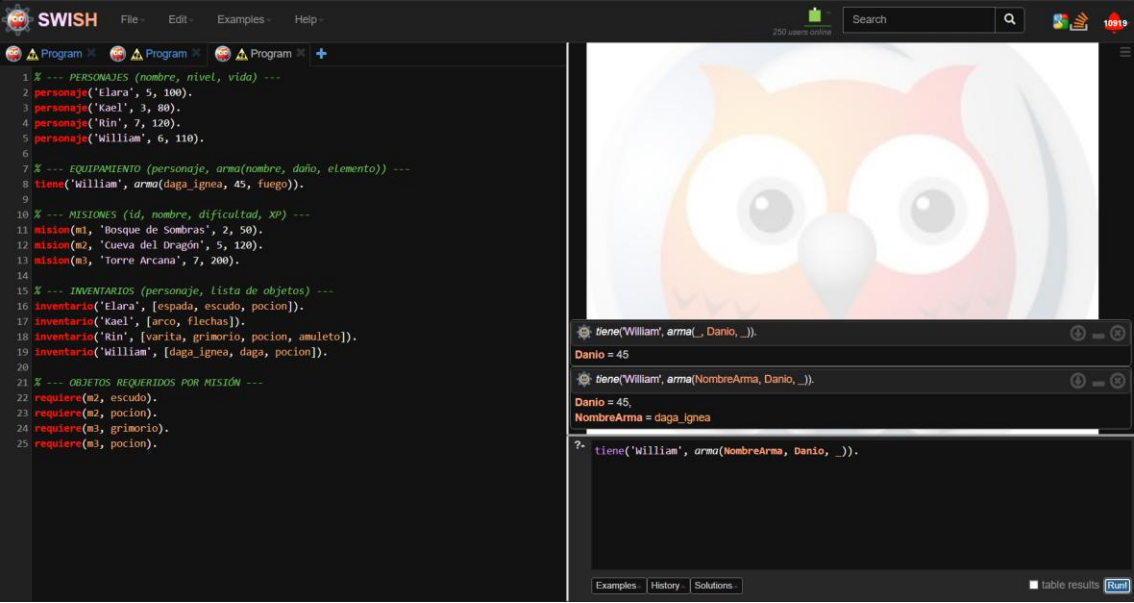


The SWISH IDE interface displays a program with the following code:

```
1 % --- PERSONAJES (nombre, nivel, vida) ---
2 personaje('Elara', 5, 100).
3 personaje('Kael', 3, 80).
4 personaje('Rin', 7, 120).
5 personaje('William', 6, 110).
6
7 % --- EQUIPAMIENTO (personaje, arma(nombre, daño, elemento)) ---
8 tiene('William', arma(daga_ignea, 45, fuego)).
9
10 % --- MISIONES (id, nombre, dificultad, xp) ---
11 mision(m1, 'Bosque de Sombras', 2, 50).
12 mision(m2, 'Cueva del Dragón', 5, 120).
13 mision(m3, 'Torre Arcana', 7, 200).
14
15 % --- INVENTARIOS (personaje, lista de objetos) ---
16 inventario('Elara', [espada, escudo, pocion]).
17 inventario('Kael', [arco, flechas]).
18 inventario('Rin', [varita, grimorio, pocion, amuleto]).
19 inventario('William', [daga_ignea, daga, pocion]).
20
21 % --- OBJETOS REQUERIDOS POR MISIÓN ---
22 requiere(m2, escudo).
23 requiere(m2, pocion).
24 requiere(m3, grimorio).
25 requiere(m3, pocion).
```

The right panel shows the execution results:

```
tiene('William', arma(_ , Danilo, _)).
Danilo = 45
personaje('William', Nivel, _).
Nivel = 6
? personaje('William', Nivel, _).
```

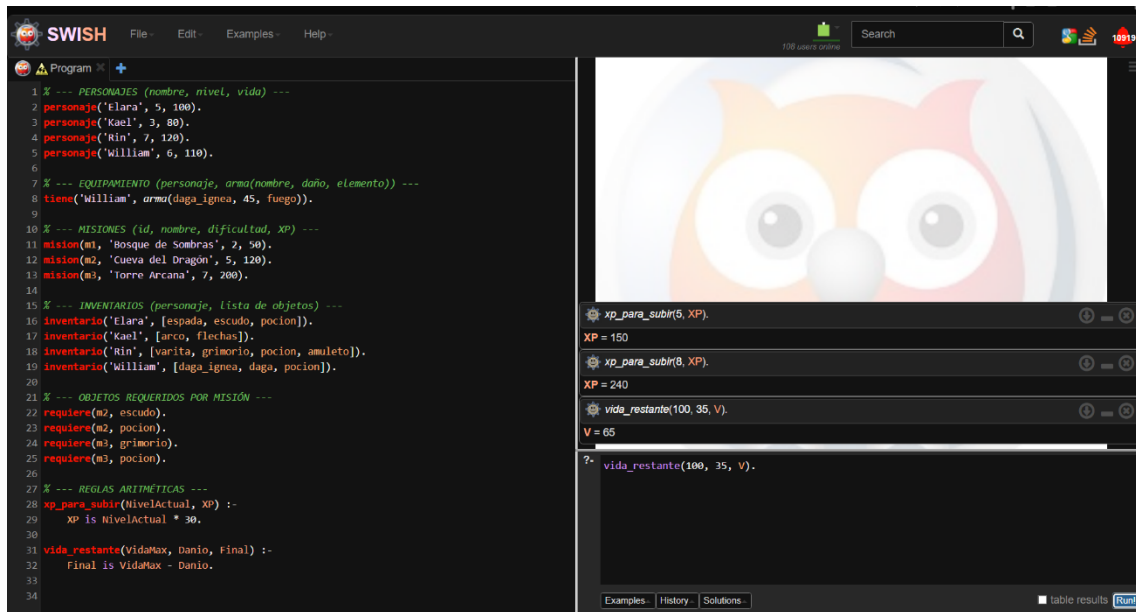


The SWISH IDE interface displays the same program as above. The right panel shows the execution results for the `tiene` function:

```
tiene('William', arma(NombreArma, Danilo, _)).
Danilo = 45
NombreArma = daga_ignea
? tiene('William', arma(NombreArma, Danilo, _)).
```

Clase 10/09/2026

IS prolog



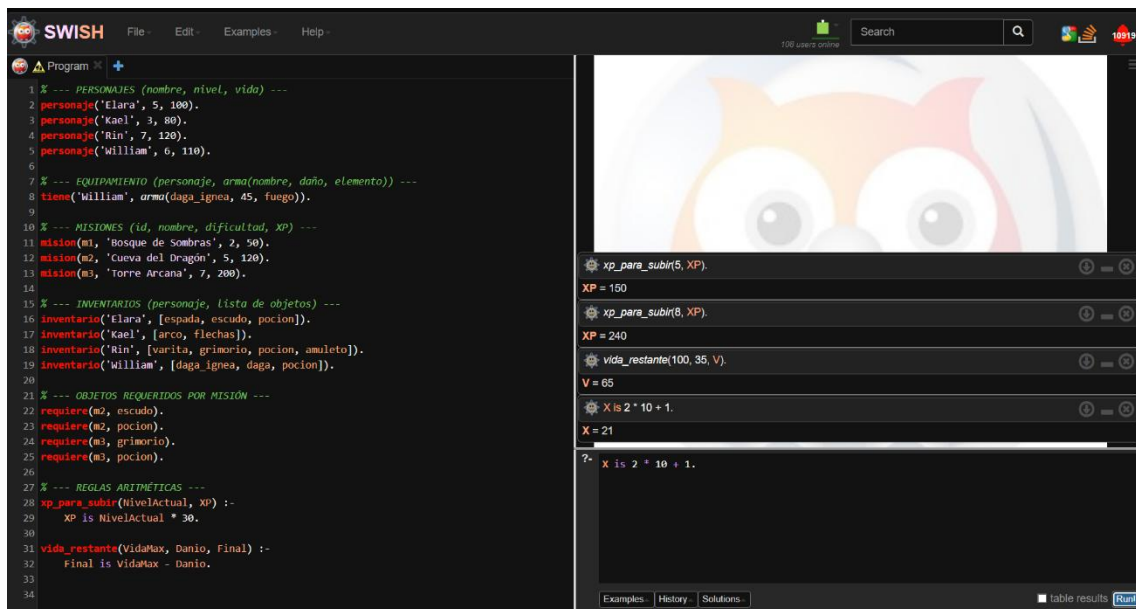
The screenshot shows the SWISH Prolog IDE interface. The left pane contains a Prolog program with the following code:

```
1 % --- PERSONAJES (nombre, nivel, vida) ---
2 personaje('Elara', 5, 100).
3 personaje('Kael', 3, 80).
4 personaje('Rin', 7, 120).
5 personaje('William', 6, 110).
6
7 % --- EQUIPAMIENTO (personaje, arma(nombre, daño, elemento)) ---
8 tiene('William', arma(daga_ignea, 45, fuego)).
9
10 % --- MISIONES (id, nombre, dificultad, XP) ---
11 mision(m1, 'Bosque de Sombras', 2, 50).
12 mision(m2, 'Cueva del Dragón', 5, 120).
13 mision(m3, 'Torre Arcana', 7, 200).
14
15 % --- INVENTARIOS (personaje, lista de objetos) ---
16 inventario('Elara', [espada, escudo, pocion]).
17 inventario('Kael', [arco, flechas]).
18 inventario('Rin', [varita, grimorio, pocion, amuleto]).
19 inventario('William', [daga_ignea, daga, pocion]).
20
21 % --- OBJETOS REQUERIDOS POR MISIÓN ---
22 requiere(m2, escudo).
23 requiere(m2, pocion).
24 requiere(m3, grimorio).
25 requiere(m3, pocion).
26
27 % --- REGLAS ARITMÉTICAS ---
28 xp_para_subir(NivelActual, XP) :-
29     XP is NivelActual * 30.
30
31 vida_restante(VidaMax, Danio, Final) :-
32     Final is VidaMax - Danio.
33
34
```

The right pane displays a character image of a stylized owl. Below the image, the following variables and their values are shown:

- `xp_para_subir(5, XP).` XP = 150
- `xp_para_subir(8, XP).` XP = 240
- `vida_restante(100, 35, V).` V = 65
- `vida_restante(100, 35, V).`

At the bottom, there are tabs for 'Examples', 'History', and 'Solutions', and a 'table results' button.



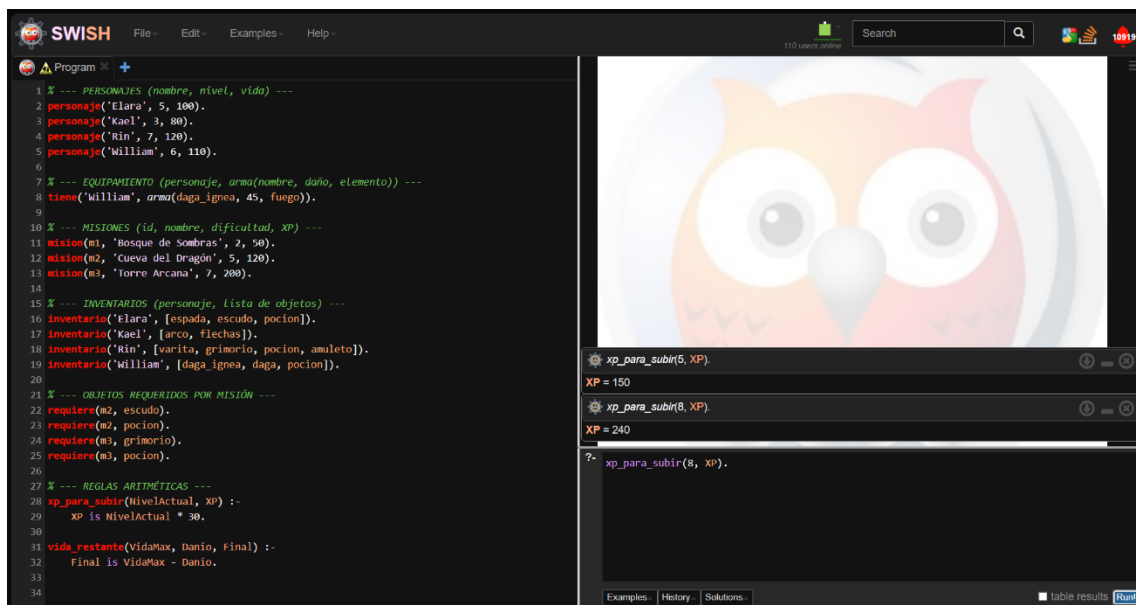
The screenshot shows the SWISH Prolog IDE interface. The left pane contains a Prolog program with the following code:

```
1 % --- PERSONAJES (nombre, nivel, vida) ---
2 personaje('Elara', 5, 100).
3 personaje('Kael', 3, 80).
4 personaje('Rin', 7, 120).
5 personaje('William', 6, 110).
6
7 % --- EQUIPAMIENTO (personaje, arma(nombre, daño, elemento)) ---
8 tiene('William', arma(daga_ignea, 45, fuego)).
9
10 % --- MISIONES (id, nombre, dificultad, XP) ---
11 mision(m1, 'Bosque de Sombras', 2, 50).
12 mision(m2, 'Cueva del Dragón', 5, 120).
13 mision(m3, 'Torre Arcana', 7, 200).
14
15 % --- INVENTARIOS (personaje, lista de objetos) ---
16 inventario('Elara', [espada, escudo, pocion]).
17 inventario('Kael', [arco, flechas]).
18 inventario('Rin', [varita, grimorio, pocion, amuleto]).
19 inventario('William', [daga_ignea, daga, pocion]).
20
21 % --- OBJETOS REQUERIDOS POR MISIÓN ---
22 requiere(m2, escudo).
23 requiere(m2, pocion).
24 requiere(m3, grimorio).
25 requiere(m3, pocion).
26
27 % --- REGLAS ARITMÉTICAS ---
28 xp_para_subir(NivelActual, XP) :-
29     XP is NivelActual * 30.
30
31 vida_restante(VidaMax, Danio, Final) :-
32     Final is VidaMax - Danio.
33
34
```

The right pane displays a character image of a stylized owl. Below the image, the following variables and their values are shown:

- `xp_para_subir(5, XP).` XP = 150
- `xp_para_subir(8, XP).` XP = 240
- `vida_restante(100, 35, V).` V = 65
- `X is 2 * 10 + 1.` X = 21
- `X is 2 * 10 + 1.`

At the bottom, there are tabs for 'Examples', 'History', and 'Solutions', and a 'table results' button.



The screenshot shows the SWISH Prolog IDE interface. The left pane contains a Prolog program with the following code:

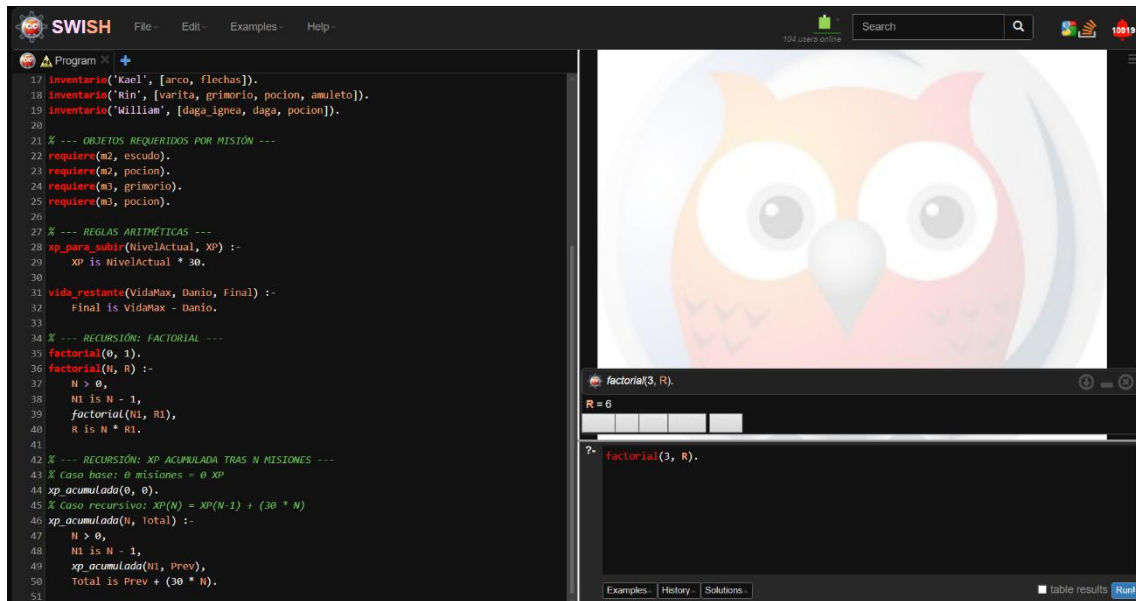
```
1 % --- PERSONAJES (nombre, nivel, vida) ---
2 personaje('Elara', 5, 100).
3 personaje('Kael', 3, 80).
4 personaje('Rin', 7, 120).
5 personaje('William', 6, 110).
6
7 % --- EQUIPAMIENTO (personaje, arma(nombre, daño, elemento)) ---
8 tiene('William', arma(daga_ignea, 45, fuego)).
9
10 % --- MISIONES (id, nombre, dificultad, XP) ---
11 mision(m1, 'Bosque de Sombras', 2, 50).
12 mision(m2, 'Cueva del Dragón', 5, 120).
13 mision(m3, 'Torre Arcana', 7, 200).
14
15 % --- INVENTARIOS (personaje, lista de objetos) ---
16 inventario('Elara', [espada, escudo, pocion]).
17 inventario('Kael', [arco, flechas]).
18 inventario('Rin', [varita, grimorio, pocion, amuleto]).
19 inventario('William', [daga_ignea, daga, pocion]).
20
21 % --- OBJETOS REQUERIDOS POR MISIÓN ---
22 requiere(m2, escudo).
23 requiere(m2, pocion).
24 requiere(m3, grimorio).
25 requiere(m3, pocion).
26
27 % --- REGLAS ARITMÉTICAS ---
28 xp_para_subir(NivelActual, XP) :-
29     XP is NivelActual * 30.
30
31 vida_restante(VidaMax, Danio, Final) :-
32     Final is VidaMax - Danio.
33
34
```

The right pane displays a character image of a stylized owl. Below the image, the following variables and their values are shown:

- `xp_para_subir(5, XP).` XP = 150
- `xp_para_subir(8, XP).` XP = 240
- `xp_para_subir(8, XP).`

At the bottom, there are tabs for 'Examples', 'History', and 'Solutions', and a 'table results' button.

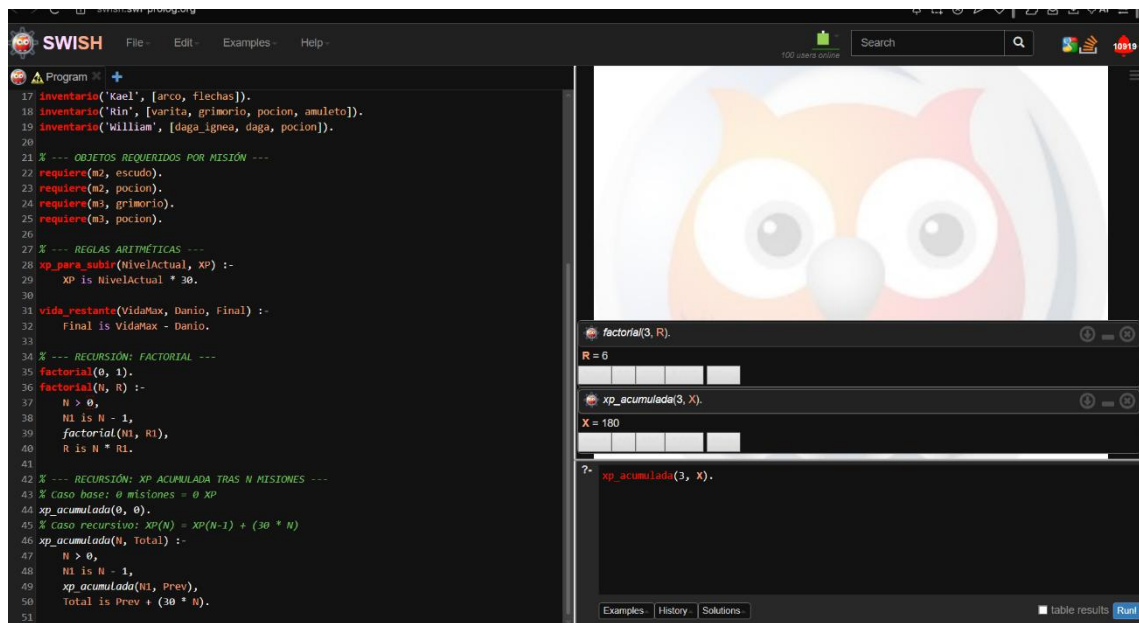
Recursion factorial y XP Acumulada tras N misiones



The screenshot shows the SWISH IDE interface. The left pane contains a program with the following code:

```
17 inventario('Kael', [arco, flechas]).
18 inventario('Rin', [varita, grimorio, pocion, amuleto]).
19 inventario('William', [daga_ignea, daga, pocion]).
20
21 % --- OBJETOS REQUERIDOS POR MISIÓN ---
22 requiere(m2, escudo).
23 requiere(m2, pocion).
24 requiere(m3, grimorio).
25 requiere(m3, pocion).
26
27 % --- REGLAS ARITMÉTICAS ---
28 xp_para_subir(NivelActual, XP) :-
29     XP is NivelActual * 30.
30
31 vida_restante(VidaMax, Danio, Final) :-
32     Final is VidaMax - Danio.
33
34 % --- RECURSIÓN: FACTORIAL ---
35 factorial(0, 1).
36 factorial(N, R) :-
37     N > 0,
38     N1 is N - 1,
39     factorial(N1, R1),
40     R is N * R1.
41
42 % --- RECURSIÓN: XP ACUMULADA TRAS N MISIONES ---
43 % Caso base: 0 misiones = 0 XP
44 xp_acumulada(0, 0).
45 % Caso recursivo: XP(N) = XP(N-1) + (30 * N)
46 xp_acumulada(N, Total) :-
47     N > 0,
48     N1 is N - 1,
49     xp_acumulada(N1, Prev),
50     Total is Prev + (30 * N).
51
```

The right pane displays a large, stylized owl image. Below the image, the execution console shows the command `factorial(3, R).` with the result `R = 6`. The console also shows the command `?- factorial(3, R).` and a `Run` button.

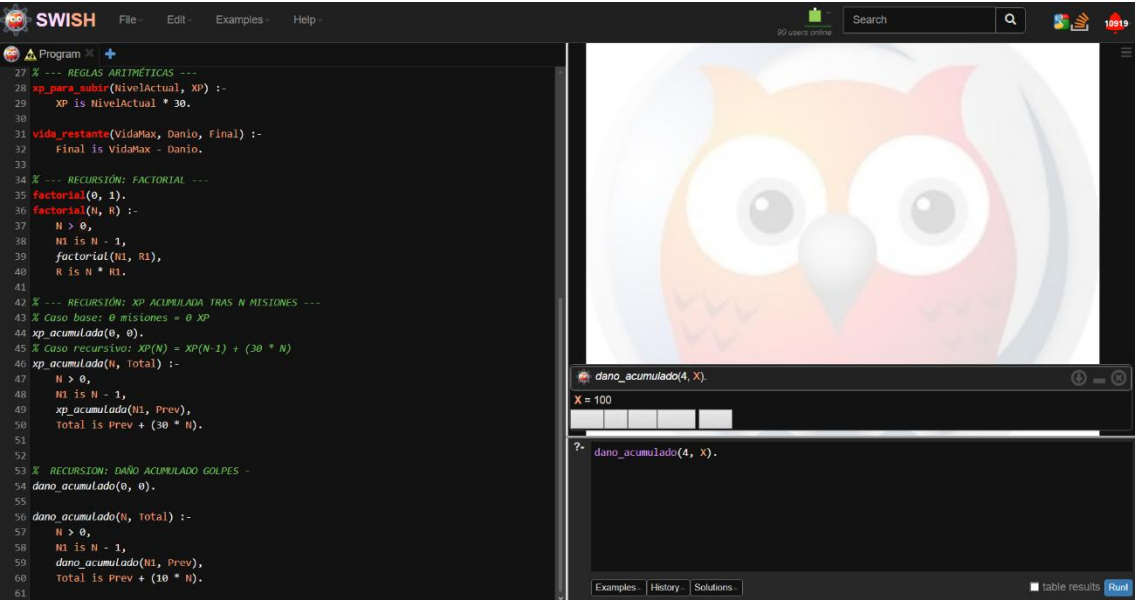


The screenshot shows the SWISH IDE interface. The left pane contains a program with the following code:

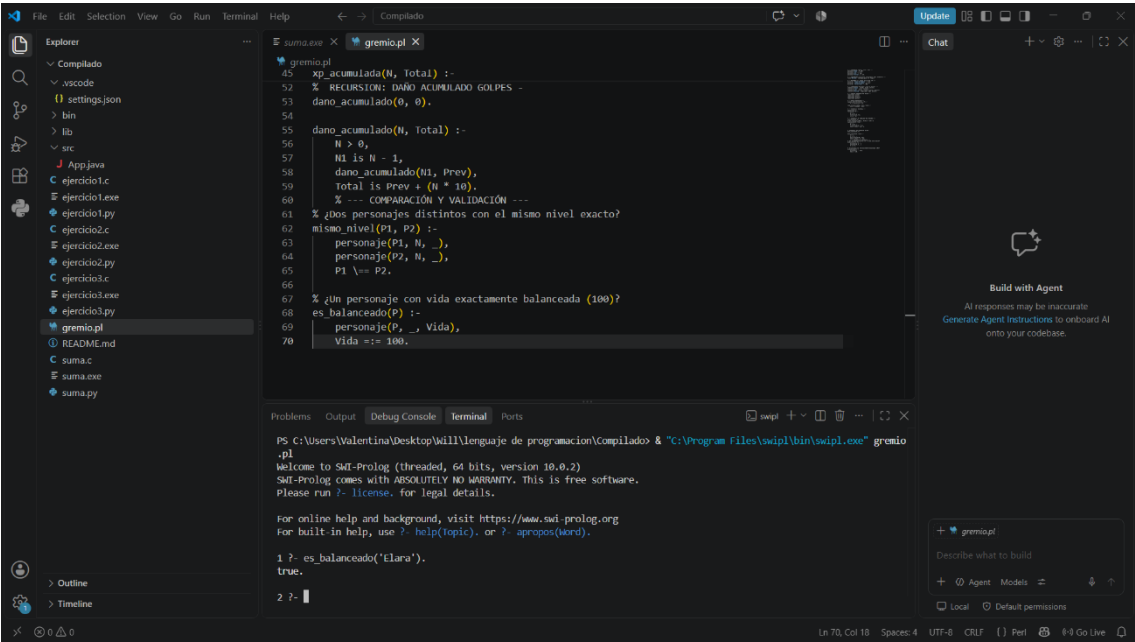
```
17 inventario('Kael', [arco, flechas]).
18 inventario('Rin', [varita, grimorio, pocion, amuleto]).
19 inventario('William', [daga_ignea, daga, pocion]).
20
21 % --- OBJETOS REQUERIDOS POR MISIÓN ---
22 requiere(m2, escudo).
23 requiere(m2, pocion).
24 requiere(m3, grimorio).
25 requiere(m3, pocion).
26
27 % --- REGLAS ARITMÉTICAS ---
28 xp_para_subir(NivelActual, XP) :-
29     XP is NivelActual * 30.
30
31 vida_restante(VidaMax, Danio, Final) :-
32     Final is VidaMax - Danio.
33
34 % --- RECURSIÓN: FACTORIAL ---
35 factorial(0, 1).
36 factorial(N, R) :-
37     N > 0,
38     N1 is N - 1,
39     factorial(N1, R1),
40     R is N * R1.
41
42 % --- RECURSIÓN: XP ACUMULADA TRAS N MISIONES ---
43 % Caso base: 0 misiones = 0 XP
44 xp_acumulada(0, 0).
45 % Caso recursivo: XP(N) = XP(N-1) + (30 * N)
46 xp_acumulada(N, Total) :-
47     N > 0,
48     N1 is N - 1,
49     xp_acumulada(N1, Prev),
50     Total is Prev + (30 * N).
51
```

The right pane displays a large, stylized owl image. Below the image, the execution console shows the command `factorial(3, R).` with the result `R = 6`. The console also shows the command `xp_acumulada(3, X).` with the result `X = 180`. The console also shows the command `?- xp_acumulada(3, X).` and a `Run` button.

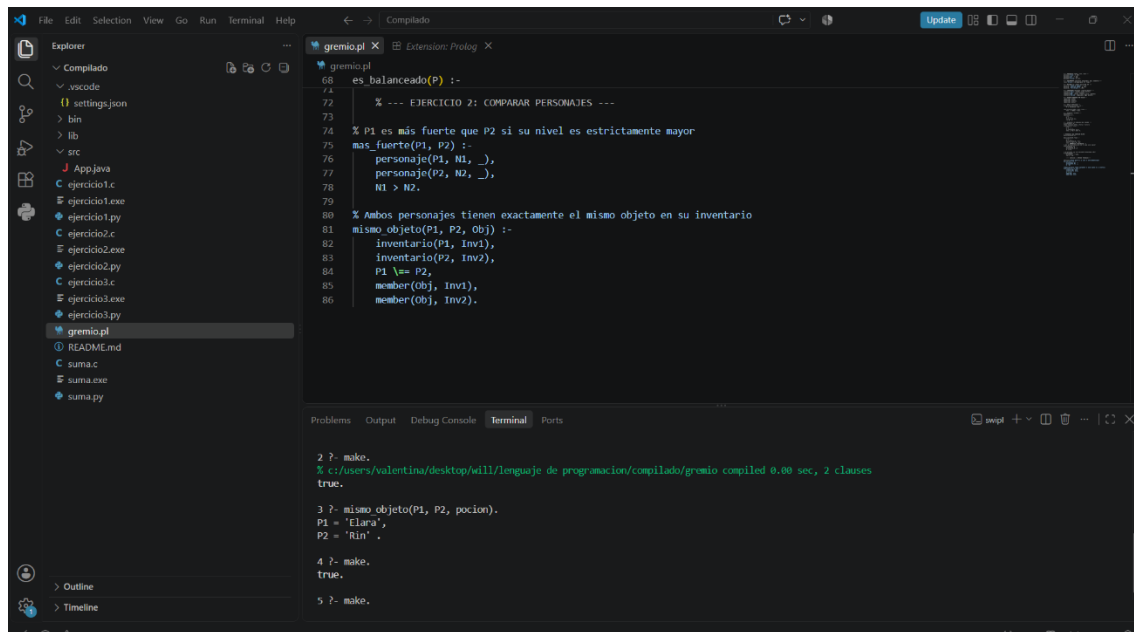
Ejercicio en clase, daño acumulado tras N golpes



Detectar Coincidencias y validar datos



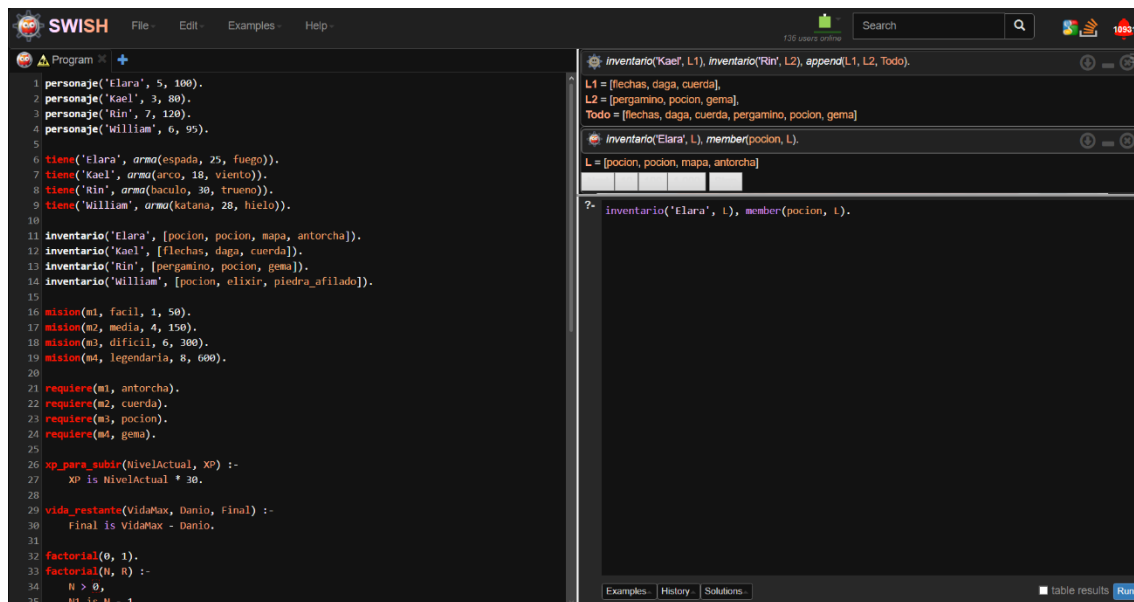
Ejercicio 2 compara personajes del gremio



```
File Edit Selection View Go Run Terminal Help
gremio.pl Extension: Prolog
68 es_balanceado(P) :-
69     % --- EJERCICIO 2: COMPARAR PERSONAJES ---
70
71     % P1 es más fuerte que P2 si su nivel es estrictamente mayor
72     mas_fuerte(P1, P2) :-
73         personaje(P1, N1, _),
74         personaje(P2, N2, _),
75         N1 > N2.
76
77     % Ambos personajes tienen exactamente el mismo objeto en su inventario
78     mismo_objeto(P1, P2, Obj) :-
79         inventario(P1, Inv1),
80         inventario(P2, Inv2),
81         P1 \== P2,
82         member(Obj, Inv1),
83         member(Obj, Inv2).
84
85
86
2 ?- make.
% c:/users/valentina/desktop/will/programacion/compilado/gremio compiled 0.00 sec, 2 clauses
true.
3 ?- mismo_objeto(P1, P2, pocion).
P1 = 'Elara',
P2 = 'Rin'.
4 ?- make.
true.
5 ?- make.
```

Clase 15/09/2026

Memebre y append sobre nuestros inventarios



```
SWISH File Edit Examples Help
135 users online Search
Program
1 personaje('Elara', 5, 100).
2 personaje('Kael', 3, 80).
3 personaje('Rin', 7, 120).
4 personaje('William', 6, 95).
5
6 tiene('Elara', arma(espada, 25, fuego)).
7 tiene('Kael', arma(arco, 18, viento)).
8 tiene('Rin', arma(baculo, 30, trueno)).
9 tiene('William', arma(katana, 28, hielo)).
10
11 inventario('Elara', [pocion, pocion, mapa, antorcha]).
12 inventario('Kael', [flechas, daga, cuerda]).
13 inventario('Rin', [pergamino, pocion, gema]).
14 inventario('William', [pocion, elixir, piedra_afilado]).
15
16 mision(m1, facil, 1, 50).
17 mision(m2, media, 4, 150).
18 mision(m3, dificil, 6, 300).
19 mision(m4, legendaria, 8, 600).
20
21 requiere(m1, antorcha).
22 requiere(m2, cuerda).
23 requiere(m3, pocion).
24 requiere(m4, gema).
25
26 xp_para_subir(NivelActual, XP) :-
27     XP is NivelActual * 30.
28
29 vida_restante(VidaMax, Danio, Final) :-
30     Final is VidaMax - Danio.
31
32 factorial(0, 1).
33 factorial(N, R) :-
34     N > 0,
35     N1 is N - 1,
```

```
inventario('Kael', L1), inventario('Rin', L2), append(L1, L2, Todo).
L1 = [flechas, daga, cuerda],
L2 = [pergamino, pocion, gema],
Todo = [flechas, daga, cuerda, pergamino, pocion, gema]
inventario('Elara', L), member(pocion, L).
L = [pocion, pocion, mapa, antorcha]
?- inventario('Elara', L), member(pocion, L).
```

```
1 personaje('Elara', 5, 100).
2 personaje('Kael', 3, 80).
3 personaje('Rin', 7, 120).
4 personaje('William', 6, 95).
5
6 tiene('Elara', arma(espada, 25, fuego)).
7 tiene('Kael', arma(arco, 18, viento)).
8 tiene('Rin', arma(baculo, 30, trueno)).
9 tiene('William', arma(katana, 28, hielo)).
10
11 inventario('Elara', [pocion, pocion, mapa, antorcha]).
12 inventario('Kael', [flechas, daga, cuerda]).
13 inventario('Rin', [pergamino, pocion, gema]).
14 inventario('William', [pocion, elixir, piedra_afilado]).
15
16 mision(m1, facil, 1, 50).
17 mision(m2, media, 4, 150).
18 mision(m3, dificil, 6, 300).
19 mision(m4, legendaria, 8, 600).
20
21 requiere(m1, antorcha).
22 requiere(m2, cuerda).
23 requiere(m3, pocion).
24 requiere(m4, gema).
25
26 xp_para_subir(NivelActual, XP) :-
27     XP is NivelActual * 30.
28
29 vida_restante(VidaMax, Danio, Final) :-
30     Final is VidaMax - Danio.
31
32 factorial(0, 1).
33 factorial(N, R) :-
34     N > 0,
35     N1 is N - 1,
```

inventario('Kael', L1), inventario('Rin', L2), append(L1, L2, Todo).

L1 = [flechas, daga, cuerda],
L2 = [pergamino, pocion, gema],
Todo = [flechas, daga, cuerda, pergamino, pocion, gema]

?- inventario('Kael', L1), inventario('Rin', L2), append(L1, L2, Todo).

Conjugar_Accion: un pequeño motor de lenguaje

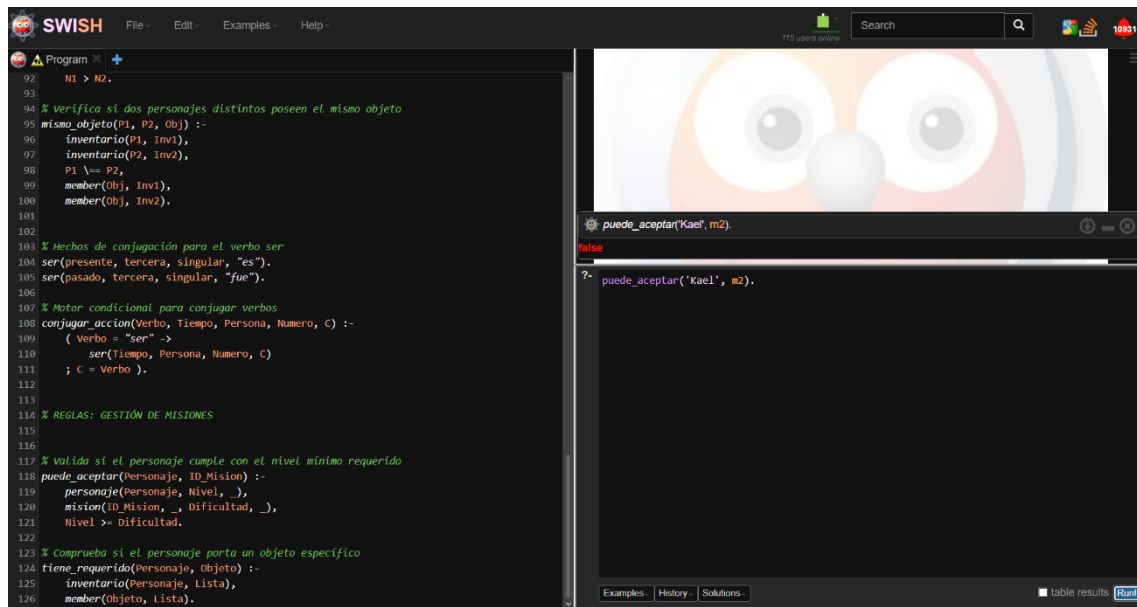
```
47 dano_acumulado(N, Total) :-
48     N > 0,
49     N1 is N - 1,
50     dano_acumulado(N1, Prev),
51     Total is Prev + (N * 10).
52
53 mismo_nivel(P1, P2) :-
54     personaje(P1, N, _),
55     personaje(P2, N, _),
56     P1 \== P2.
57
58 es_balanceado(P) :-
59     personaje(P, _, Vida),
60     Vida >= 100.
61
62 mas_fuerte(P1, P2) :-
63     personaje(P1, N1, _),
64     personaje(P2, N2, _),
65     N1 > N2.
66
67 mismo_objeto(P1, P2, Obj) :-
68     inventario(P1, Inv1),
69     inventario(P2, Inv2),
70     P1 \== P2,
71     member(Obj, Inv1),
72     member(Obj, Inv2).
73
74 % --- INFERENCIA CONDICIONAL: MOTOR DE LENGUAJE ---
75 ser(presente, tercera, singular, "es").
76 ser(pasado, tercera, singular, "fue").
77
78 conjugar_accion(Verbo, Tiempo, Persona, Numero, C) :-
79     ( Verbo = "ser" ->
80     ser(Tiempo, Persona, Numero, C)
81     ; C = Verbo ).
```

conjugar_accion("ser", presente, tercera, singular, C).

C = "es"

?- conjugar_accion("ser", presente, tercera, singular, C).

¿puede in personaje aceptar una misión?

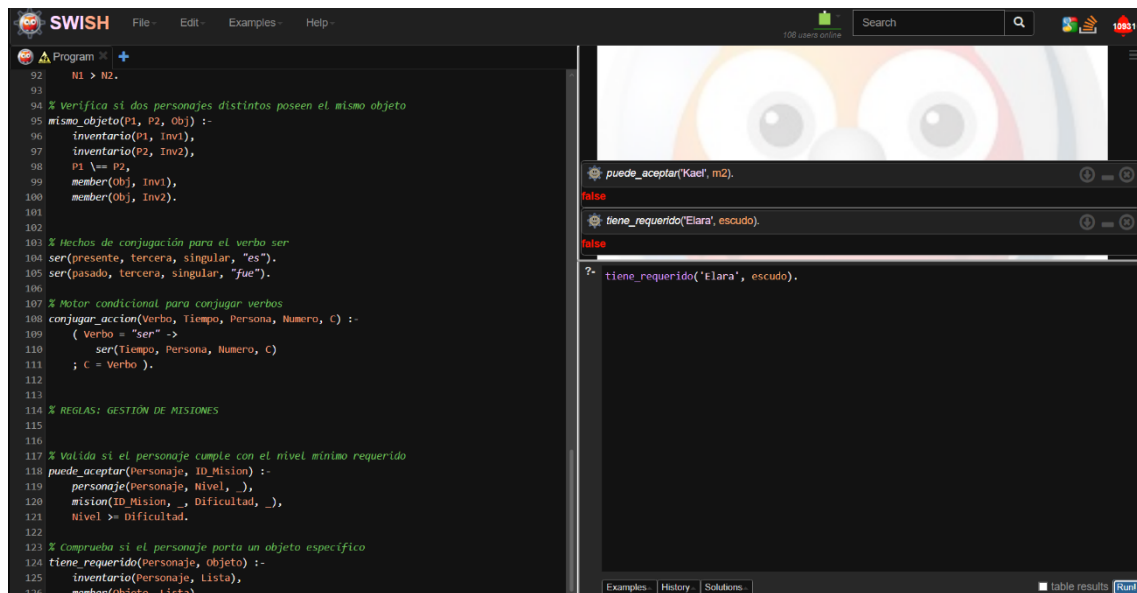


The image shows the SWISH Prolog IDE interface. The left pane contains a Prolog program with the following code:

```
92 N1 > N2.
93
94 % Verifica si dos personajes distintos poseen el mismo objeto
95 mismo_objeto(P1, P2, Obj) :-
96     inventario(P1, Inv1),
97     inventario(P2, Inv2),
98     P1 \== P2,
99     member(Obj, Inv1),
100     member(Obj, Inv2).
101
102
103 % Hechos de conjugación para el verbo ser
104 ser(presente, tercera, singular, "es").
105 ser(pasado, tercera, singular, "fue").
106
107 % Motor condicional para conjugar verbos
108 conjugar_accion(Verbo, Tiempo, Persona, Numero, C) :-
109     ( Verbo = "ser" ->
110         ser(Tiempo, Persona, Numero, C)
111     ; C = Verbo ).
112
113
114 % REGLAS: GESTIÓN DE MISIONES
115
116
117 % Valida si el personaje cumple con el nivel mínimo requerido
118 puede_aceptar(Personaje, ID_Mision) :-
119     personaje(Personaje, Nivel, _),
120     mision(ID_Mision, _, Dificultad, _),
121     Nivel >= Dificultad.
122
123 % Comprueba si el personaje porta un objeto específico
124 tiene_requerido(Personaje, Objeto) :-
125     inventario(Personaje, Lista),
126     member(Objeto, Lista).
```

The right pane shows a query window with the following content:

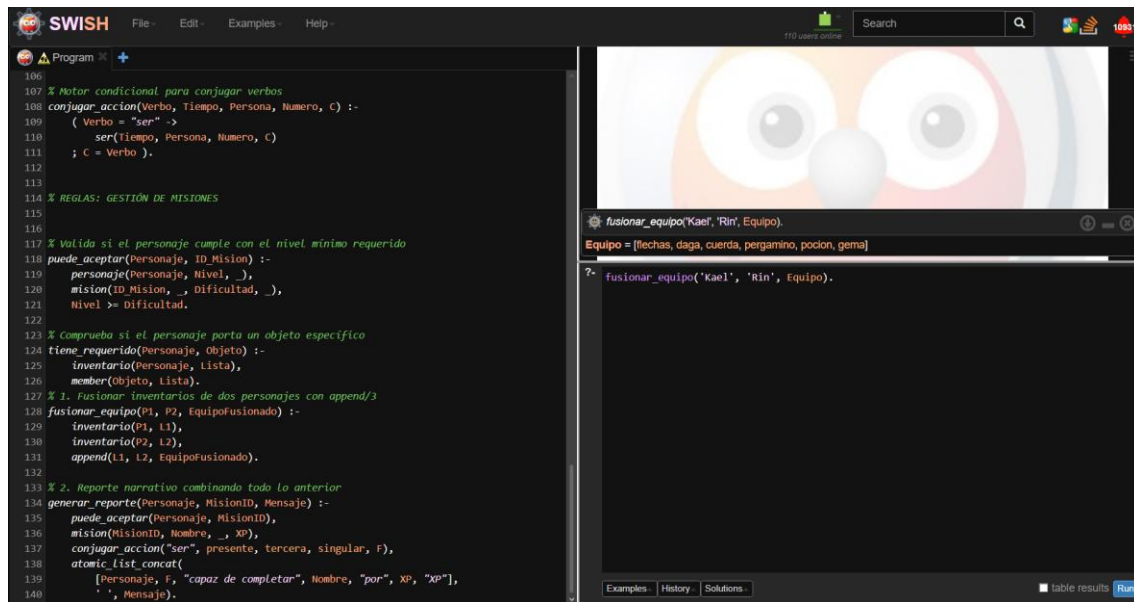
```
puede_aceptar('Kael', m2).
false
?- puede_aceptar('Kael', m2).
```



The image shows the SWISH Prolog IDE interface. The left pane contains the same Prolog program as in the first image. The right pane shows a query window with the following content:

```
puede_aceptar('Kael', m2).
false
tiene_requerido('Elara', escudo).
false
?- tiene_requerido('Elara', escudo).
```

Fusionar equipos y narrar el resultado

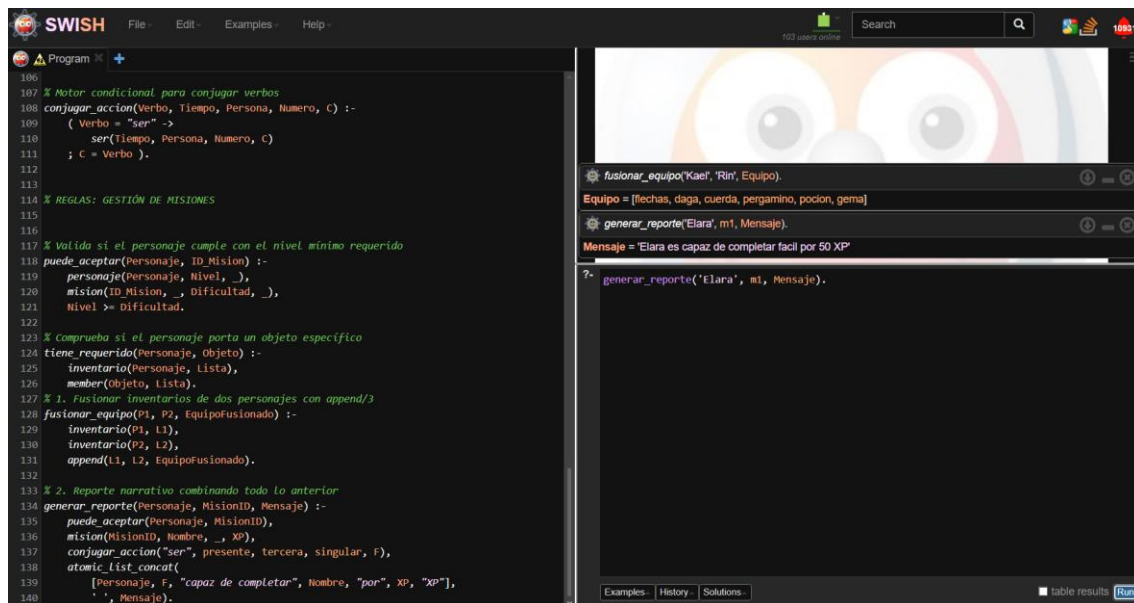


The screenshot shows the SWISH IDE interface. The left pane contains Prolog code with comments in Spanish. The right pane shows a console window with a cartoon owl logo and the execution of a query.

```
106
107 % Motor condicional para conjugar verbos
108 conjugar_accion(Verbo, Tiempo, Persona, Numero, C) :-
109     ( Verbo = "ser" ->
110         ser(Tiempo, Persona, Numero, C)
111         ; C = Verbo ).
112
113
114 % REGLAS: GESTIÓN DE MISIONES
115
116
117 % Valida si el personaje cumple con el nivel mínimo requerido
118 puede_aceptar(Personaje, ID_Mision) :-
119     personaje(Personaje, Nivel, _),
120     mision(ID_Mision, _, Dificultad, _),
121     Nivel >= Dificultad.
122
123 % Comprueba si el personaje porta un objeto específico
124 tiene_requerido(Personaje, Objeto) :-
125     inventario(Personaje, Lista),
126     member(Objeto, Lista).
127
128 % 1. Fusionar inventarios de dos personajes con append/3
129 fusionar_equipo(P1, P2, EquipoFusionado) :-
130     inventario(P1, L1),
131     inventario(P2, L2),
132     append(L1, L2, EquipoFusionado).
133
134 % 2. Reporte narrativo combinando todo lo anterior
135 generar_reporte(Personaje, MisionID, Mensaje) :-
136     puede_aceptar(Personaje, MisionID),
137     mision(MisionID, Nombre, _, XP),
138     conjugar_accion("ser", presente, tercera, singular, F),
139     atomic_list_concat(
140         [Personaje, F, "capaz de completar", Nombre, "por", XP, "XP"],
141         ' ', Mensaje).
```

Console output:

```
fusionar_equipo('Kael', 'Rin', Equipo).
Equipo = [flechas, daga, cuerda, pergamino, pocion, gema]
?- fusionar_equipo('Kael', 'Rin', Equipo).
```



The screenshot shows the SWISH IDE interface. The left pane contains Prolog code with comments in Spanish. The right pane shows a console window with a cartoon owl logo and the execution of a query.

```
106
107 % Motor condicional para conjugar verbos
108 conjugar_accion(Verbo, Tiempo, Persona, Numero, C) :-
109     ( Verbo = "ser" ->
110         ser(Tiempo, Persona, Numero, C)
111         ; C = Verbo ).
112
113
114 % REGLAS: GESTIÓN DE MISIONES
115
116
117 % Valida si el personaje cumple con el nivel mínimo requerido
118 puede_aceptar(Personaje, ID_Mision) :-
119     personaje(Personaje, Nivel, _),
120     mision(ID_Mision, _, Dificultad, _),
121     Nivel >= Dificultad.
122
123 % Comprueba si el personaje porta un objeto específico
124 tiene_requerido(Personaje, Objeto) :-
125     inventario(Personaje, Lista),
126     member(Objeto, Lista).
127
128 % 1. Fusionar inventarios de dos personajes con append/3
129 fusionar_equipo(P1, P2, EquipoFusionado) :-
130     inventario(P1, L1),
131     inventario(P2, L2),
132     append(L1, L2, EquipoFusionado).
133
134 % 2. Reporte narrativo combinando todo lo anterior
135 generar_reporte(Personaje, MisionID, Mensaje) :-
136     puede_aceptar(Personaje, MisionID),
137     mision(MisionID, Nombre, _, XP),
138     conjugar_accion("ser", presente, tercera, singular, F),
139     atomic_list_concat(
140         [Personaje, F, "capaz de completar", Nombre, "por", XP, "XP"],
141         ' ', Mensaje).
```

Console output:

```
fusionar_equipo('Kael', 'Rin', Equipo).
Equipo = [flechas, daga, cuerda, pergamino, pocion, gema]
generar_reporte('Elara', m1, Mensaje).
Mensaje = 'Elara es capaz de completar facil por 50 XP'
?- generar_reporte('Elara', m1, Mensaje).
```

SWISH

File · Edit · Examples · Help

110 users online

Search

Q

100%

Program

106

107 *% Motor condicional para conjugar verbos*

108 *conjugar_accion(Verbo, Tiempo, Persona, Numero, C) :-*

109 *(Verbo = "ser" ->*

110 *ser(Tiempo, Persona, Numero, C)*

111 *; C = Verbo).*

112

113

114 *% REGLAS: GESTIÓN DE MISIONES*

115

116

117 *% Valida si el personaje cumple con el nivel mínimo requerido*

118 *puede_aceptar(Personaje, ID Mision) :-*

119 *personaje(Personaje, Nivel, _),*

120 *mision(ID Mision, _, Dificultad, _),*

121 *Nivel >= Dificultad.*

122

123 *% Comprueba si el personaje porta un objeto específico*

124 *tiene_requerido(Personaje, Objeto) :-*

125 *inventario(Personaje, Lista),*

126 *member(Objeto, Lista).*

127 *% 1. Fusionar inventarios de dos personajes con append/3*

128 *fusionar_equipo(P1, P2, EquipoFusionado) :-*

129 *inventario(P1, L1),*

130 *inventario(P2, L2),*

131 *append(L1, L2, EquipoFusionado).*

132

133 *% 2. Reporte narrativo combinando todo lo anterior*

134 *generar_reporte(Personaje, MisionID, Mensaje) :-*

135 *puede_aceptar(Personaje, MisionID),*

136 *mision(MisionID, Nombre, _, XP),*

137 *conjugar_accion("ser", presente, tercera, singular, F),*

138 *atomic_list_concat(*

139 *[Personaje, F, "capaz de completar", Nombre, "por", XP, "XP"],*

140 *_, Mensaje).*

Call: generar_reporte('Elara',m2,_7486)

Msg = 'Elara es capaz de completar media por 150 XP'

X = 10, X == 10.

X = 10

Y = 10.

false

Z =:= 10.

Arguments are not sufficiently instantiated

in:

[1] _2334-...10

Z =:= 10.

Examples · History · Solutions

table results

Run